

An Emotionally Modulated Trading Agent As A Use Case Of The MetaMo Framework

Introduction

This document describes the trading agent use case of the MetaMo framework. MetaMo is a motivation system that combines MAGUS and OpenPSI: it appraises what happens in the environment, updates an agent's goals and modulators, scores possible actions against them, and keeps the whole state stable. The framework itself does not care what the environment is. This use case gives it a market.

The idea is simple. An agent starts with 100 in cash and trades one asset over a price series. It can buy, sell or hold, and it has no trading rule anywhere in its code. Every decision comes out of the framework: the OpenPSI appraisal turns each price move into modulator updates, the MAGUS scorer weighs the possible actions against the current goals and modulators, two subsystems with different personalities (a careful one and an eager one) have to agree on one action through the framework's consensus transition, and the stability dynamics keep every update inside the safe region. After each step the OpenPSI emotion layer reports how the agent feels, so you can watch happiness rise during a rally and fear rise during a crash while the trades happen.

For comparison there is also a baseline agent in the same market. It is a plain momentum rule, buy after a rise and sell after a fall, with none of the MetaMo machinery. Since both agents trade in exactly the same world, any difference in their behavior comes from the motivational system and nothing else.

The rest of this document explains the files and how they fit together, how the market and the adapters work, how to run the agents and the tests, how to read the charts, how to bring in your own price data including real historical data, and what results the two agents get on the included market scenarios.

Methodology

Design rule

The whole use case follows one rule: the framework is consumed unchanged, and everything specific to trading lives in the adapters. If a behavior needed fixing during development, the fix had to go into what the agent perceives or which actions it is offered, never into the framework's appraisal, scoring, consensus or stability code. The final pull request adds only new files.

Controlled market scenarios

Instead of testing on one market, the agent is evaluated on four scenarios that each stress a different situation. The vshape scenario (20 steps) is a smooth rally, crash and recovery, made to watch the motivation change with the market. The choppy scenario (20 steps) is a whipsaw that swings four to five percent around a flat level, which is the known failure case of momentum rules. The downtrend scenario (20 steps) keeps bleeding with small bounces, so nothing long-only can profit and the question becomes who loses less. The realistic scenario (50 steps) is generated once by a seeded random walk with three phases, an uptrend, a crash containing two panic days of minus 9.5 and minus 12 percent, and a choppy recovery, because real crashes contain single days far outside the normal noise. All series are fixed rather than random at runtime, so the tests, the CI and every rerun give exactly the same numbers.

A control agent in the same world

The baseline is a plain momentum rule: buy when flat after any rise above one percent, sell when holding after any fall below minus one percent. It trades in the same market file with the same portfolio rules and uses none of the MetaMo machinery. Because the world is shared, any difference in behavior between the two agents can only come from the motivational system.

Iterative design, documented honestly

The adapters were not right on the first try. In the first version the agent sold on a panic day and bought right back the next day, five times in a row, because its perception only looked at the last return, so the quiet day after a panic looked like a calm market. The fix was the remembered turbulence with decay and downside weighting, applied to both the stimulus risk and the candidates. A second problem was that the agent read as happy while sitting in cash during a crash, which made no sense; the fix was letting a flat agent's conduciveness sink with remembered turbulence. Both fixes are perception changes with named constants and comments explaining the reasoning, and after each change the full test suite and all four scenarios were rerun.

Evaluation pipeline synced with the PeTTa code

Every metric is computed inside the PeTTa run by the framework itself. Each step the agent prints one parseable atom with the scenario, price, chosen action, dominant emotion, the four

feeling intensities, valence, securing, the two overgoals and the portfolio value. The Python side never recomputes anything: `evaluate_trading.py` launches both agents through the PeTTa runner, parses the printed atoms and writes `eval/evaluation_results.json`, following the same pattern as the repository's `qwestor_eval.py`, and `plot_trading_run.py` parses the same atoms to draw the charts.

Testing

The use case carries 45 unit tests that are discovered and run by the repository's own `scripts/run-tests.py` with the PeTTa runner. They cover the market arithmetic and the no-op trades, the stimulus staying inside its bounds for extreme inputs, the turbulence memory (a quiet day after a panic day still reads as turbulent, falls count fully and rises count half), the candidate shapes and their turbulence scaling, the position-aware candidate sets, both subsystem states, and one complete framework cycle on a crash observation.

Validation that nothing is scripted

Two counterfactual experiments check that the trades really come from the motivational state. First, single MetaMo cycles were run with the same subsystems while only the perceived market changed: calm while holding gives hold, crash while holding gives sell, calm while flat gives buy, crash while flat gives hold. Second, the identical realistic series was run with the ambition subsystem replaced by a second cautious one: that agent made zero trades in 49 steps and finished at exactly 100.0, while the normal agent trades three times and finishes at 112.35. A fixed price rule could not change its trades when only the agent's personality changes.

Real historical data

To check that the pipeline is not limited to generated series, real Apple weekly closes from May 2015 to July 2016 were converted into a scenario with `make_scenario.py` and run through the same pipeline. This exposed a calibration point worth documenting: the perception gains were tuned for panic-day-sized moves, while real stock corrections grind down in three to six percent weeks, so the agent under-reacted with the default gain (final 82.5) and behaved sensibly once the risk gain was matched to the asset's volatility (final 95.71, against a market that lost 23.6 percent and a baseline at 101.42). The conclusion is that the risk gain is a units choice that must match the asset and timescale, and the committed defaults suit markets with panic-scale moves such as daily crypto data.

Reproducibility

Every reported number reproduces identically on every run, and the full pipeline was verified independently on two machines: the development environment and the author's own system, both showing 45 of 45 tests passing and the same four final values per agent.

How the Agent Works

The agent runs the same loop for every step of the price series: it looks at the market, the framework appraises what it sees, the framework picks an action, the trade is executed, and the emotion layer reports how the agent feels. This section goes through that loop with the actual formulas used at each stage.

Step 1. What the agent sees

The adapter first computes a remembered turbulence, because real market volatility comes in clusters and one quiet day inside a crash should not feel calm:

$$turbulence(t) = \max(w(r) \cdot |r| \cdot RISK_GAIN, TURBULENCE_DECAY \cdot turbulence(t - 1))$$

where r is the last return, $RISK_GAIN = 6$, $TURBULENCE_DECAY = 0.75$, and $w(r)$ is 1.0 for falls but only 0.5 for rises, because falls are what make a market feel dangerous. From this the adapter builds the framework's four-part stimulus. Novelty is $|r| \cdot 8$, so big moves feel new. Risk is the remembered turbulence, plus 0.25 extra when the agent is holding and the price just fell, because losses on an open position hurt more. Conduciveness depends on the position: while holding it is $0.5 + 6r$ (a rise is good, a fall is bad), and while flat it is

$0.5 - 1.2 \cdot turbulence$, because an agent in cash is not losing money but a crashing market is still a scary place. Effort is a constant 0.1. Everything is clipped to $[0, 1]$. This stimulus is the only thing the agent ever learns about the market.

Step 2. Appraisal

The OpenPSI appraisal turns the stimulus into changes of the six modulators. For example, the valence and securing updates are

$$\Delta Valence = 0.75 \cdot Conduciveness + 0.25 \cdot BenignNovelty - (0.55 \cdot Risk + 0.15 \cdot Effort)$$

$$\Delta Securing = 0.80 \cdot Risk + 0.20 \cdot Effort - (0.30 \cdot BenignNovelty + 0.10 \cdot Conduciveness)$$

with similar equations for arousal, approach, resolution and threshold, and each modulator is then bounded back into $[0, 1]$ by the framework's sigmoid bound. This is where the market becomes feeling: a crash step has low conduciveness and high risk, so valence collapses and securing rises; a rally does the opposite.

Step 3. Where emotion meets decision

The MAGUS scorer rates each offered action against the current goals and modulators. The framework's own relevance table decides which modulator weighs which goal:

$$\begin{array}{ll}
 g_{Curio} \leftarrow Arousal & g_{Help} \leftarrow Resolution \\
 g_{Novel} \leftarrow Approach & g_{Soc} \leftarrow \frac{Valence + Approach}{2} \\
 g_{Ethic} \leftarrow \frac{Threshold + Securing}{2} & g_{Self} \leftarrow \frac{Approach + Resolution}{2}
 \end{array}$$

This table is the exact mechanism that makes the trading emotional. Our buy candidate correlates with g_{Curio} and g_{Novel} , so buying is strong when arousal and approach are high, which is how the agent feels in a rally. Our sell candidate correlates with g_{Ethic} , so selling is strong when threshold and securing are high, which is how the agent feels after danger. The full score of a candidate is

$$Score = BaseScore - (RiskPenalty + ConflictPenalty) + GrowthReward$$

$$RiskPenalty = \lambda_{Ind} \cdot g_{Ind} \cdot CautionSignal \cdot RiskEstimate$$

$$GrowthReward = \lambda_{Trans} \cdot g_{Trans} \cdot GrowthSignal \cdot NormalizedGrowth$$

$$\lambda_{Ind} = \lambda_{Trans} = 0.5$$

$$CautionSignal = \frac{Threshold + Securing}{2}$$

$$GrowthSignal = \frac{Arousal + Approach}{2}$$

A frightened and highly secured state increases the penalty on risky actions, while an aroused and approach-oriented state increases the reward for growth-related actions.

Step 4. Consensus of two personalities

The agent is two subsystems: prudence (individuation-dominant, high caution modulators) and ambition (transcendence-dominant, high arousal and approach). Each scores the candidates from its own appraised state, and the framework combines them:

$$ConsensusScore = \frac{Score_A + Score_B}{2} - 0.25 \cdot |Score_A - Score_B|$$

The penalty on disagreement is what gives the agent its patience: an action only wins when both personalities can live with it, which is why small wobbles never trigger a sale but a real crash does.

Step 5. Stability

Every state update is damped by $C_CONTRACT = 0.9$, checked against the contractive update law, and projected back into the safe region if the individuation goal would fall below $THETA_SAFE = 0.3$, with boundary caution raised near the edge. The agent's goals therefore drift with its trades (selling nudges g_{Ind} up, buying nudges g_{Trans} up through each action's δG) but can never run away.

Step 6. The emotion readout

After each step the feeling layer computes the intensity $I(E)$ of each of the four emotions by comparing the current modulators against that emotion's target levels from the emotion table in `openpsi/config.metta`, using the fuzzy membership functions, and summing the contributions over the five emotion dimensions. The dominant emotion is the one whose intensity exceeds the threshold $EMOTION_GAMMA = 0.6$; if none does, the agent reports neutral. Every log line carries the dominant emotion and the full (happy, sad, angry, fear) vector, and these are what the charts show.

One clarification

The emotion *label* is a readout, not an input: `magusScore` never reads the word "happy" or "fear". What drives the decisions are the modulators and goals and those same modulators are what the feeling layer classifies into emotions. So the decisions and the emotions are two views of one affective state. That is the precise sense in which this agent trades on emotion: when the chart shows fear rising and the agent selling, both come from the same appraised state, computed by the same framework, in the same step.

Experiments and findings

We ran four experiments in total: the main benchmark against the baseline on four market scenarios, an action-flip test, a personality counterfactual, and a run on real historical data. On top of these, 45 unit tests cover the parts in isolation, and the full pipeline was reproduced on two different machines with identical numbers.

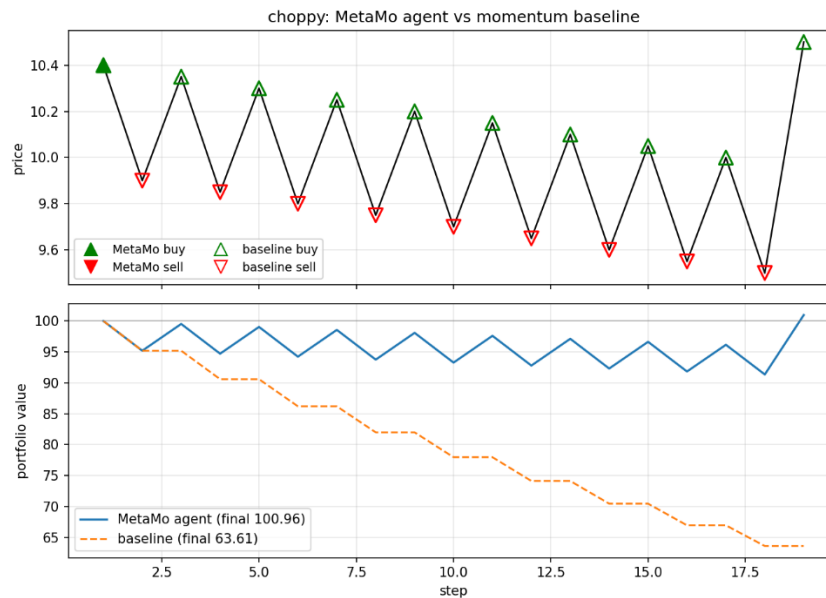
Experiment 1. Benchmark against the momentum baseline

Both agents traded the four scenarios starting with 100. The results:

- realistic: agent 112.35, baseline 121.62
- vshape: agent 137.76, baseline 155.62
- choppy: agent 100.96, baseline 63.61
- downtrend: agent 82.47, baseline 72.07
- total: agent 433.54, baseline 412.92

The clearest finding is the choppy market. The baseline buys every pop and sells every dip, nine losing round trips in a row, and loses 36 percent of its capital. The agent buys once at 10.4 and holds through the whole whipsaw, because the consensus between prudence and ambition never clears the bar on small wobbles. The baseline wins the two smooth trending series, and it should, because a one percent threshold rule is close to ideal on a clean trend. The agent wins the downtrend and the total. On the realistic series the agent buys the uptrend at 99.15, sells at 92.3 on the first panic day of minus 9.5 percent, sits in cash through the minus 12 percent day and the bottom at 70.09 while its valence falls from 0.77 to 0.20 and its fear intensity reaches its maximum for the run, then re-enters at 71.45 and rides the recovery to plus 12.35 percent through a market that fell 37 percent peak to trough.

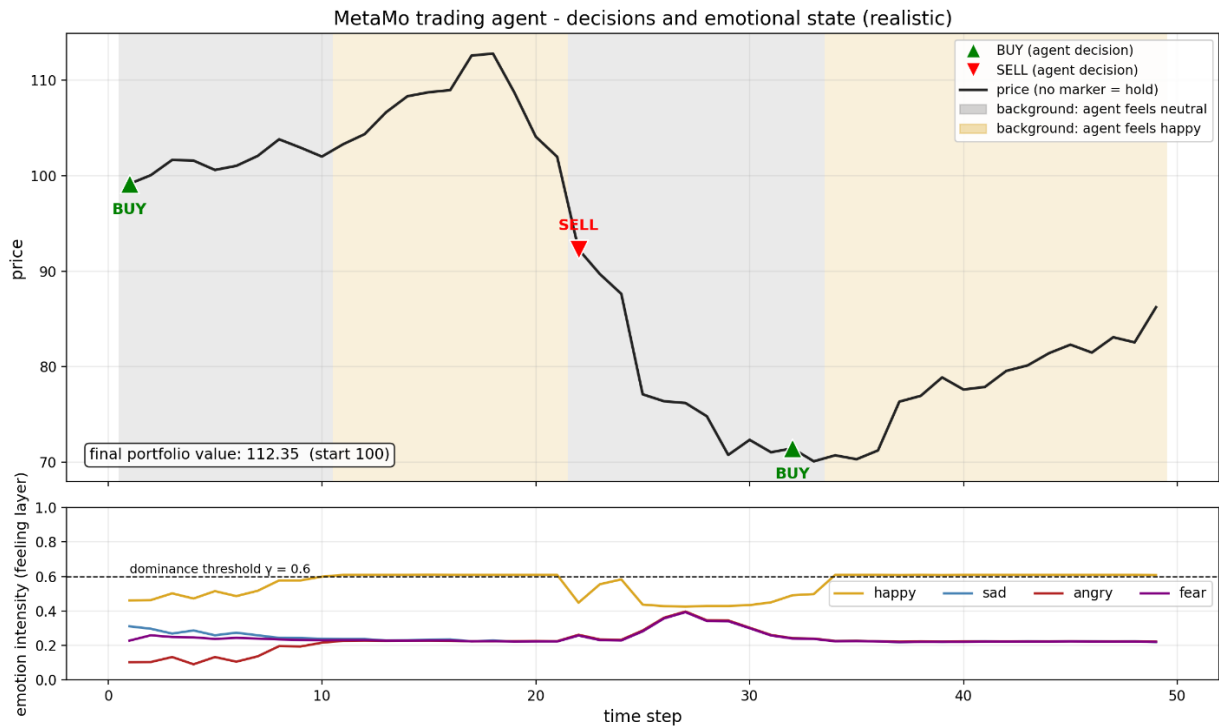
Figures for this experiment: the four trading_agent_vs_baseline charts (one per scenario, solid markers are the agent, hollow are the baseline, with both portfolio value curves below), the four trading_agent_press charts (price with the agent's decisions, its dominant feeling as background color, and the four emotion intensity curves), and the four trading_agent_dynamics charts (modulators, overgoals and emotion step by step). The choppy comparison chart and the realistic press chart carry the two main findings.



The top panel is the choppy market itself: the price zigzags between roughly 9.5 and 10.4 with no trend at all, then pops to 10.5 on the last step. The markers are the trades. The solid green triangle at step 1 is the MetaMo agent's only trade. It buys at 10.4 and never trades again. The hollow triangles are the baseline: a hollow green buy on almost every peak, a hollow red sell on almost every dip, all the way across. You can literally see the momentum rule buying every local top and selling every local bottom, because that is what buy after a rise, sell after a fall means in an alternating market.

The bottom panel is what those choices cost. The orange dashed line is the baseline's portfolio value, a staircase going down. Each step down is one round trip: buy near 10.3, get shaken out near 9.8, and lock in about a five percent loss, nine times in a row, ending at 63.61. The flat parts of the staircase are the moments it sits in cash between mistakes. The blue line is the agent. It wobbles up and down because it is holding the asset, so its value moves with the price, but it never sells into a dip, so nothing is ever locked in, and the final pop carries it out at 100.96.

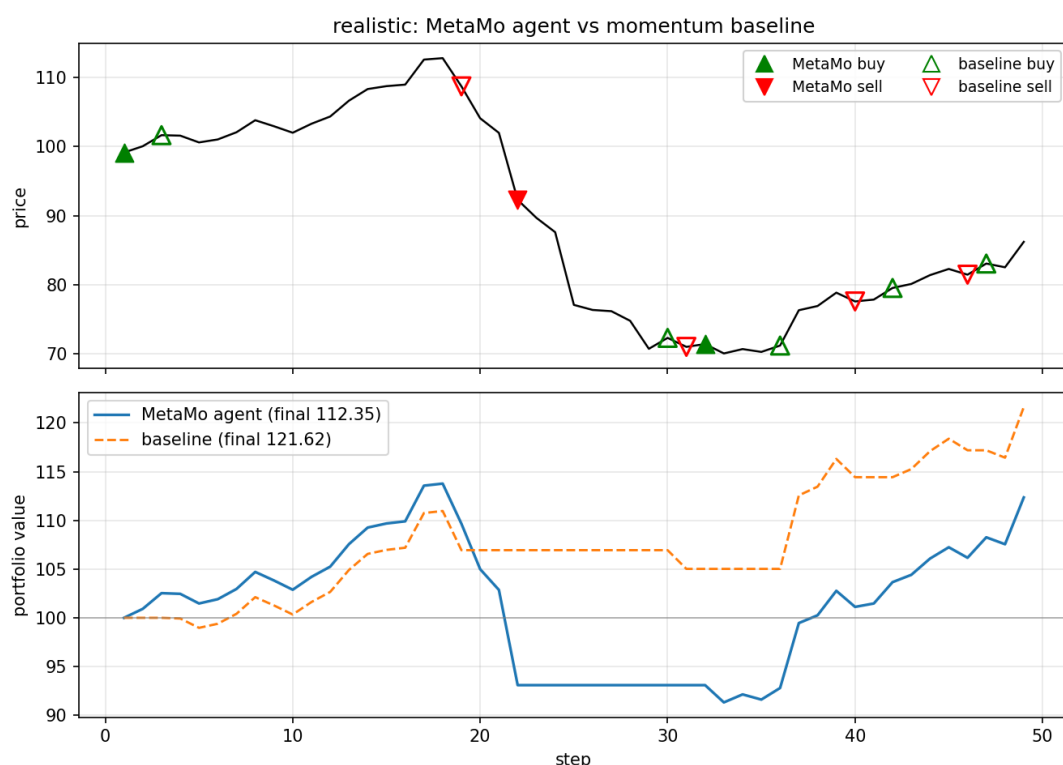
Why the agent holds is the interesting part. After its first buy, every wobble raises the perceived turbulence a little, which makes both remaining options less attractive at once, and an action only executes when both subsystems agree on it. Prudence and ambition never agree that a four percent dip in a trendless market is worth panicking over, so the consensus keeps landing on hold. The patience is not a rule anyone wrote. It falls out of the disagreement penalty in the consensus formula.



The top panel is a 50 step market that looks like a real chart: an uptrend to about 113, a crash with two panic days down to 70, then a choppy recovery to 86. The black line is the price, the triangles are the agent's actual decisions, and the background color of every step is the agent's dominant feeling at that moment, gold for happy and gray for neutral. No color or marker is placed by hand. Everything is parsed from the agent's own run log.

Reading it left to right, the agent buys at 99.15 on the second step, while the background is still gray, so it enters calmly, not euphorically. As the rally develops the background turns gold: the rising prices read as conducive, valence climbs, and happy crosses the dominance threshold. At step 22 comes the first panic day of minus 9.5 percent. The agent sells at 92.3 and the gold ends right there. Through the entire rest of the crash, the minus 12 percent day and the grind to the bottom at 70.09, the background stays gray. The agent is in cash, so it loses nothing, but a violent market still reads as hostile, so it stays uneasy instead of happy, and its valence sinks to 0.20 in this stretch. Then, as the remembered turbulence fades and the recovery starts, it buys at 71.45, close to the bottom, and the background turns gold again as it rides the recovery. The final value is 112.35, a plus 12 percent result through a market that fell 37 percent peak to trough.

The bottom panel shows where the colors come from: the four raw emotion intensities from the OpenPSI feeling layer, computed fresh every step. The gold happy curve sits above the dashed dominance threshold ($\gamma = 0.6$) during the rally, collapses when the crash starts, and stays down while the agent waits in cash. Meanwhile the purple fear curve rises through the crash and peaks near the bottom around steps 25 to 29, exactly when the market is at its worst, then eases off as the agent re-enters and happy recovers above the line. This is the panel to point at when someone asks whether the emotions are real. The fear peak was not drawn, it was computed by the same framework state that decided the trades.

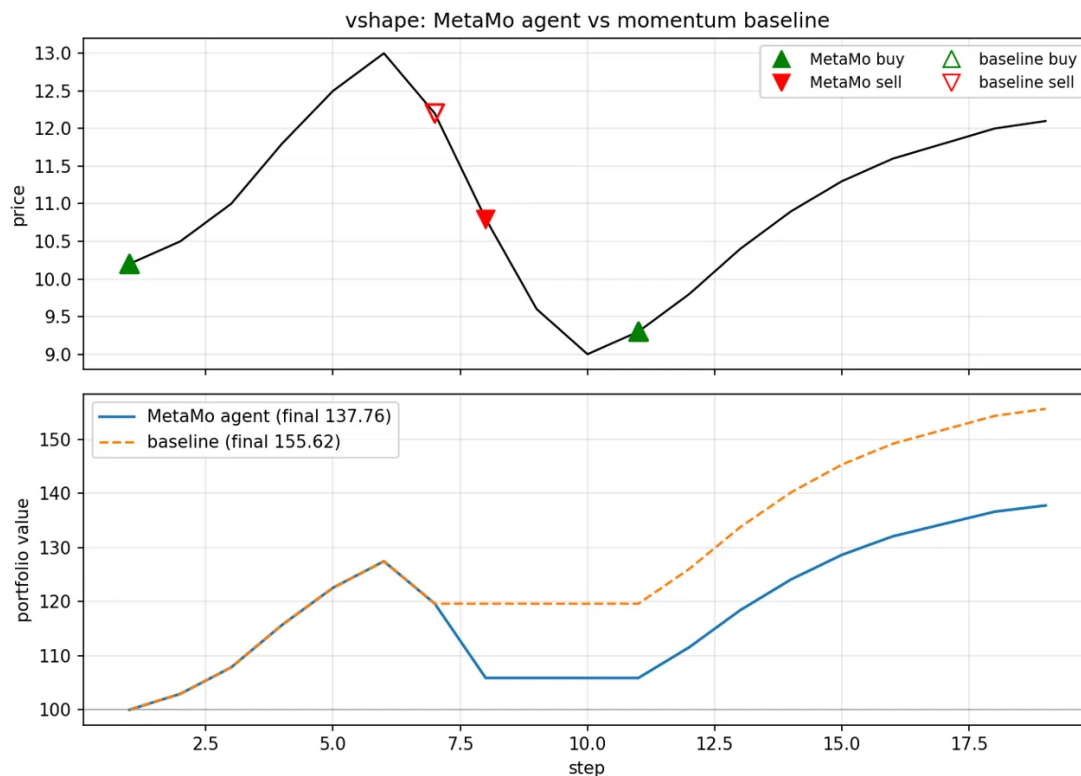


The top panel shows the trades. The agent, with solid markers, makes exactly three decisions: buy at 99.15 on step 1, sell at 92.3 on the first panic day, and buy back at 71.45 two steps off the exact bottom. The baseline, with hollow markers, tells a different story. It buys at about 101.7 early in the rally and rides up, which works fine at first. Then look closely at step 19, right after the 112.8 peak. The very first red day of the crash was already minus 3.6 percent, far past its minus 1 percent trigger, so the baseline sold at about 108.7, just three percent off the top. That single lucky exit is almost the entire reason it wins this chart. After that, watch the bottom of the crash around steps 30 to 36: a hollow buy at about 72 and a hollow sell at about 70.7. It got whipsawed at the bottom and locked in a small loss, the same disease that destroyed it in the choppy market, just milder. Through the recovery it churns, selling at about 78, rebuying at about 79.5, selling at about 81.5 and rebuying at about 83. Each of those round trips sells low and rebuys higher, leaking a little every time.

The bottom panel shows what it cost and paid. Both curves climb the rally together. The orange baseline flattens at about 107 from step 19, which is it sitting in cash after the lucky top exit. Then you can see the small dip near step 31 from the bottom whipsaw, and a staircase up through the recovery churn to 121.62. The blue agent peaks near 114, gives some back waiting for real evidence of danger, flattens at about 93 after selling on the panic day, dips briefly after re-entering at 71.45 since the market touched 70.09 right after, then climbs cleanly to 112.35.

The honest reading is that the nine point gap comes down to one number, exit at 108.7 versus exit at 92.3. The baseline's hair trigger happened to fire three percent from the top of this

particular series, while the agent by design does not react to a single red day and waits for a genuine panic. On this market the hair trigger paid. In the choppy market the identical hair trigger lost 36 percent. The baseline also showed its weakness here too, in the bottom whipsaw and the recovery churn, it just got paid enough at the top to afford them. And both agents beat simply holding the market, which ends at 86.

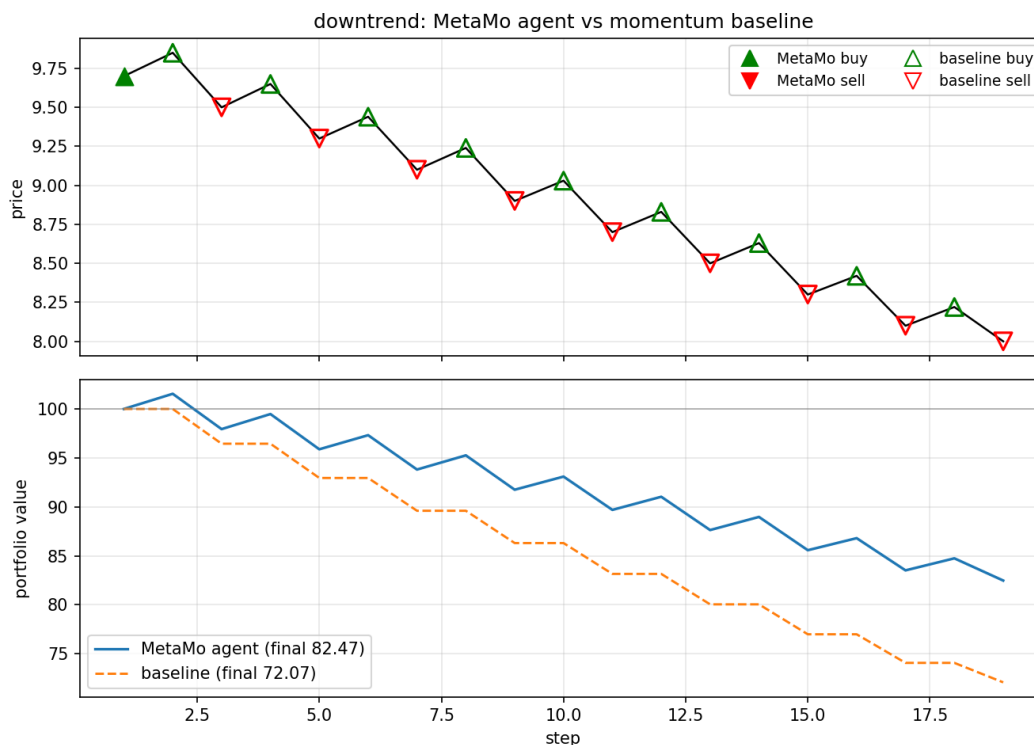


The top panel is a textbook V: a smooth rally from 10 to 13, a sharp crash to 9, and a steady recovery to 12.1. Notice something subtle about the markers. At the buy points, 10.2 at step 1 and 9.3 at step 11, you only see solid triangles. That is because both agents bought at the exact same steps and prices, so the hollow baseline markers are hidden underneath the solid ones. The two agents differ at exactly one moment, the exit. The baseline's hollow sell sits at 12.2, the first down day after the peak, which was already minus 6.2 percent and far past its minus 1 percent trigger. The agent's solid sell sits one step later at 10.8. It needed the second, truly violent day of minus 11.5 percent before both subsystems agreed the market had turned dangerous.

The bottom panel shows the curves literally on top of each other up the rally to about 127, and parallel again through the whole recovery. Everything that separates 155.62 from 137.76 happens in that one step between the two sell markers. The baseline parked at about 119.6, the agent at about 105.9, and since both re-entered together at 9.3, the gap just carried through to the end.

The honest reading is that this chart and the choppy chart are the same trigger judged by two different markets. The baseline's hair trigger fires on the first minus 1 percent day every time.

On a clean V that reflex is perfect. It exits six percent off the top and wins by 18 points. In the whipsaw the identical reflex fires nine times on noise and loses 36 percent. The agent's consensus needs stronger evidence before acting, which costs one step here and saves the account there. Neither is smarter on every series. The difference is that the agent's caution is a property of its motivational state, not a fixed number, and both beat simply holding, which ends around 118.6.



The top panel shows a market that bleeds from 9.7 to 8.0 in a sawtooth. Every leg is a 3.5 percent drop followed by a 1.5 percent bounce. The agent has exactly one marker on the whole chart, the solid buy at 9.7 on step 1. After that it holds all the way down and no sell ever fires. The baseline's markers are on every tooth: a hollow green buy on top of every little bounce and a hollow red sell at the bottom of every drop. Look at the pattern. It is buying at 9.83 and selling at 9.5, buying at 9.63 and selling at 9.3, all the way down. That is the choppy market disease again, just tilted downhill. Every bounce reads as momentum to buy and every drop shakes it out at a lower price.

The bottom panel shows the blue line as a scaled copy of the price. The agent holds, so its value simply tracks the market down to 82.47, losing exactly what the market lost, 17.5 percent. The orange staircase falls faster. Each flat segment is the baseline briefly in cash, each drop is a fresh round trip that bought high and sold low, and that friction stacks on top of the market's decline to end at 72.07, a 28 percent loss in a market that only fell 17.5 percent.

The honest reading has two findings. First, the agent wins by ten points not by doing something clever but by refusing to churn. When the market offers no winning move, doing nothing is the best available move, and that patience is the consensus mechanism again, since the two subsystems never agree that a 3.5 percent dip is worth panicking over. Second, there is a

limitation here. The agent never sold on the way down, because the 3.5 percent legs keep the remembered turbulence around 0.2, far below the level where both subsystems agree to exit, so it rode the whole bleed. That is the same under-reaction later measured on real Apple weekly data with the default gains. Grinding declines do not produce panic scale turbulence, so the sell behavior is calibrated for crashes, not for slow bleeds, and the risk gain is the knob that fixes it.

Experiment 2. Action-flip test.

The action flip test checks that the agent's decisions are computed and not scripted. We ran four single MetaMo cycles with the exact same subsystems and the exact same code, and only changed what the agent perceives: a calm market with a small move and low turbulence, or a crash with an 11.5 percent drop and high turbulence, once while holding the asset and once while sitting in cash.

The four results are the four cells of the figure. Calm while holding gives hold, since there is no reason to leave a quiet market. Crash while holding gives sell, since the danger pushes the caution modulators up and the safety goal takes over. Calm while flat gives buy, since low turbulence lets the curiosity and novelty side win. Crash while flat gives hold, since even an agent in cash treats a crashing market as a place to stay out of.

The point of the experiment is that nothing else changed between the four cases. There is no rule in the code that maps prices to actions, so if the behavior were static the agent would have picked the same action every time. Instead the same agent chose four different actions for four different perceived situations, which shows the choice really comes from the appraisal, the motivational state and the consensus.

Action flip test: same agent, same code, only the perceived market changes

	calm market (small move, low turbulence)	crash (-11.5% day, high turbulence)
holding the asset	HOLD keeps riding the calm market	SELL protects the position from danger
flat (in cash)	BUY enters the calm market	HOLD stays safely in cash

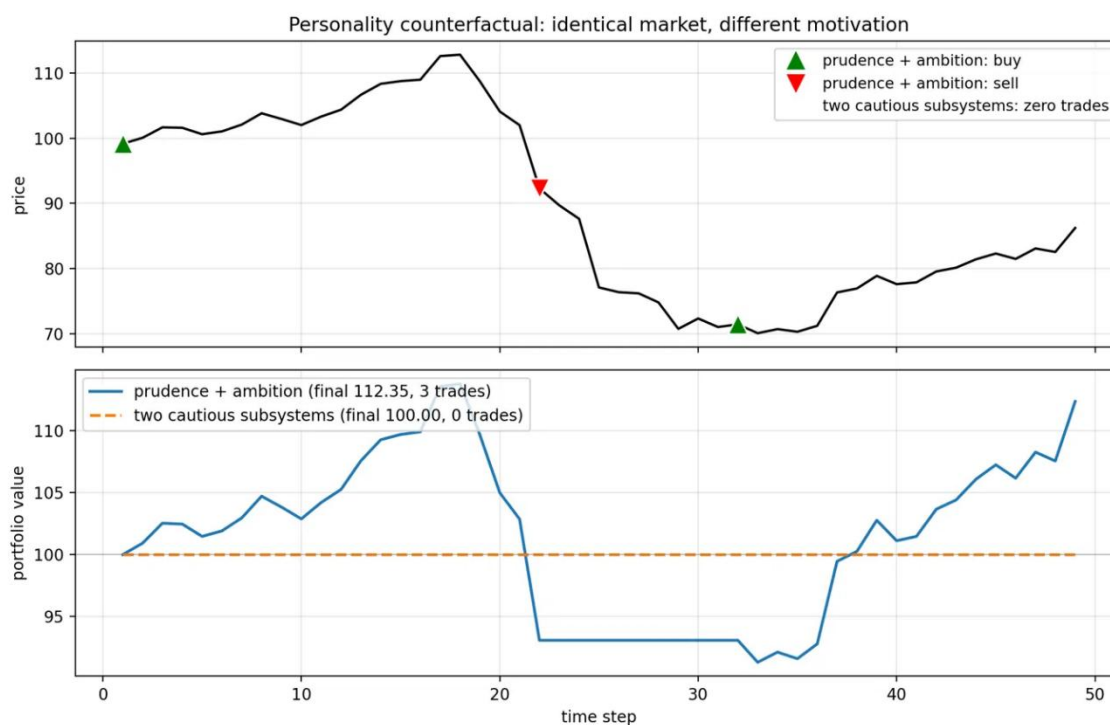
Experiment 3. Personality counterfactual

The personality counterfactual asks the opposite question of the action flip test. There the market changed and the agent stayed the same, here the market stays the same and the agent's personality changes. We ran the realistic price series twice. The first run is the normal agent with its two subsystems, prudence and ambition. The second run replaces ambition with a second copy of the cautious prudence profile, so the agent is made of two careful subsystems and nothing in it wants growth. Everything else is identical: the same prices, the same code, the same framework.

The top panel shows the price with the normal agent's three trades on it, the buy at 99.15, the sell at 92.3 on the first panic day and the re-entry at 71.45 near the bottom. The all-cautious agent has no markers at all, because it never traded once in 49 steps.

The bottom panel shows what that means for the portfolio. The normal agent's value rides the rally, steps aside during the crash and climbs through the recovery to 112.35. The all-cautious agent's value is a flat line at exactly 100.00 from the first step to the last. Without the ambition subsystem there is nothing to push the buy score high enough, so the consensus lands on hold every single time and the agent just sits in cash forever.

This is the strongest evidence that the trades come from the motivational system. A fixed price rule cannot change its trades when only the agent's personality changes, because the prices it reacts to are identical. Here the identical market produced three trades for one personality and zero trades for another, so the desire to trade lives in the motivation, not in the price data.



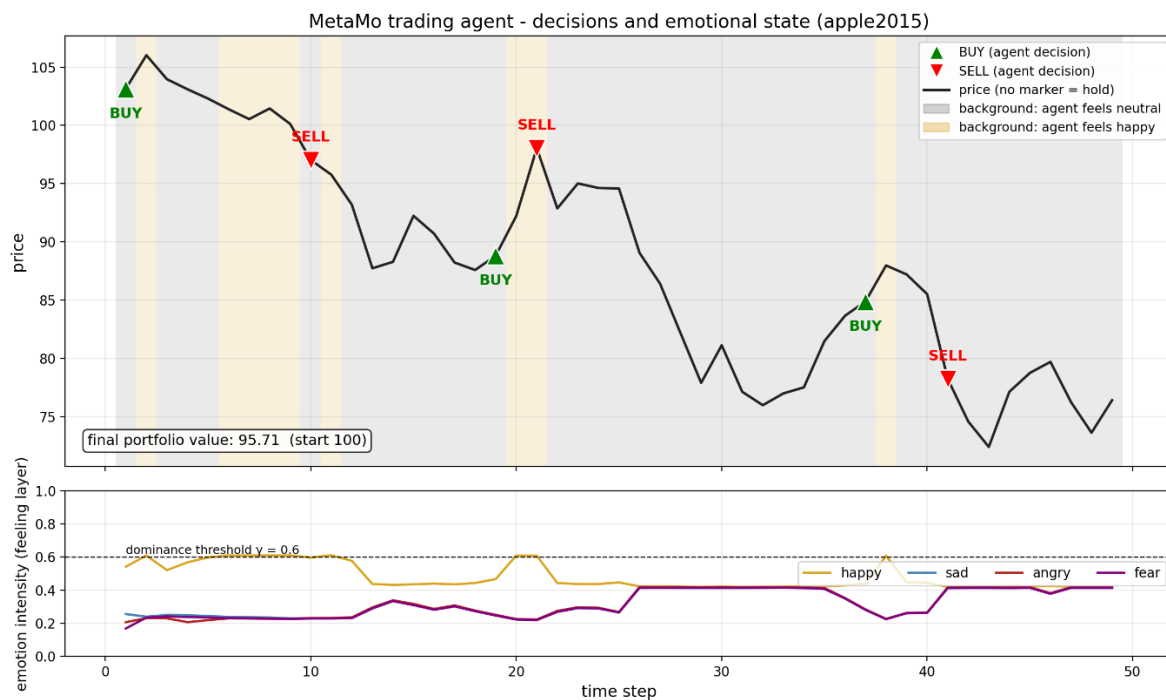
Experiment 4. Real historical data

Experiment 4 tests the pipeline on real historical data instead of a generated series. The prices are real Apple weekly closes from May 2015 to July 2016, taken from a public dataset, converted with `make_scenario.py` and normalized to start at 100. This window contains two real market events, the August 2015 correction and the January 2016 selloff, and the market itself lost 23.6 percent over it. The chart shows the run with the risk gain matched to the asset, `RISK_GAIN` 12 instead of the default 6, because weekly stock moves are smaller than the panic day scale the defaults were tuned for.

The top panel reads left to right. The agent buys at 103.15 in the first week and enjoys a short happy stretch as the price pushes toward 106. As the slow August 2015 slide develops it sells at 97.03, only about eight percent off the top, and steps aside. When the market bounces it re-enters at 88.82, rides the bounce up, and sells again at 98.05, which turns out to be almost exactly the top of that bounce. That single round trip earned about ten percent. From there the long January 2016 decline begins and the background stays gray for most of it, the agent sitting in cash and reading the market as hostile. It tries the recovery attempt at 84.9, and when that attempt fails it exits at 78.25 rather than riding the next leg down. It finishes at 95.71, a loss of about four percent in a market that lost 23.6 percent.

The bottom panel shows the emotions behind those moves. The happy curve holds near the dominance threshold through the early stretch and the bounce, then falls away for good once the big decline starts. The fear curve does the opposite. It rises during the first slide, eases during the bounce, and then climbs to around 0.42 and stays there through the entire long decline, week after week. That sustained elevated fear during a real bear market, computed fresh at every step from the framework state, is the clearest sign in the whole project that the emotional readout tracks reality.

The experiment also produced the project's main calibration finding. With the default gains the same agent under-reacted on this data and finished at 82.5, because a three to six percent weekly grind never reaches the turbulence level that panic days produce. Matching the risk gain to the asset's volatility fixed the behavior without touching anything else. The lesson is that the risk gain works like a unit choice that must fit the asset and timescale, and the committed defaults suit markets with panic sized daily moves. For reference, the momentum baseline finished this window at 101.42, edging the agent on final value, while buy and hold finished at 76.4.



How to run and test the agent yourself

What you need. You need three things: the MetaMo repository, a PeTTa installation (SWI-Prolog with the Janus bridge), and a Python that has numpy and matplotlib. If your numpy lives inside a virtual environment, activate it first, because the framework's math helpers call numpy through the Python bridge. In all the commands below, replace `/path/to/PeTTa/run.sh` with wherever your PeTTa run script actually is.

Step 1. Run the tests. From the repository root:

```
python3 scripts/run-tests.py --root usecase/metamo-trading-agent --petta-runner /path/to/PeTTa/run.sh
```

This uses the repository's own test runner and should end with 45/45 test markers passed. If it fails with "No module named numpy", your Python environment is not active.

Step 2. Run both agents and keep the logs.

```
cd usecase/metamo-trading-agent
```

```
sh /path/to/PeTTa/run.sh trading_baseline.metta | tee baseline_log.txt
```

```
sh /path/to/PeTTa/run.sh trading_agent.metta | tee agent_log.txt
```

The baseline finishes in under a minute. The MetaMo agent takes a few minutes because every step is a full framework cycle, so do not worry when it looks slow. You will see one log line per step and a FINAL line per scenario. The expected finals are: agent 137.76, 100.96, 82.47 and 112.35 for vshape, choppy, downtrend and realistic, and baseline 155.62, 63.61, 72.07 and 121.62. These numbers reproduce exactly on every run and every machine.

Step 3. Make the charts.

```
python3 plot_trading_run.py agent_log.txt baseline_log.txt figures
```

This writes three charts per scenario into a figures folder: the comparison against the baseline, the internal dynamics, and the presentation chart with the agent's decisions and emotional state.

Step 4. Run the evaluation pipeline.

```
python3 evaluate_trading.py run /path/to/PeTTa/run.sh
```

This launches both agents through PeTTa, parses their printed atoms, prints a summary table and writes `eval/evaluation_results.json`. If you already have logs from step 2 you can skip the rerun with:

```
python3 evaluate_trading.py logs agent_log.txt baseline_log.txt
```

Testing with your own data. Any price history works as long as you can export it as a CSV file with a column of closing prices. For example, download the historical data for a stock or a cryptocurrency from Yahoo Finance as a CSV. Then, from the use case folder:

```
python3 make_scenario.py csv yourfile.csv mymarket Close
```

Replace Close with the actual name of the price column if it differs. The script normalizes the first price to 100 and prints a scenario equation. Paste that equation into trading_market.metta next to the existing scenarios, then add this line at the bottom of trading_agent.metta:

```
!(startTrader mymarket)
```

And if you want the baseline comparison, add the matching line at the bottom of trading_baseline.metta:

```
!(baselineLoop mymarket (initialPortfolio) 1)
```

Then repeat steps 2 and 3 and your market shows up in the logs and the charts like any other scenario. Keep your series around 50 to 100 points, since every point is a full framework cycle and very long series just take proportionally longer.

One tip about timescales. The default perception gains are tuned for markets with panic sized moves, meaning single steps of ten percent or more, which is normal for daily cryptocurrency data. If your data moves more gently, for example weekly stock closes where a bad week is three to six percent, the agent will under-react with the defaults. In that case raise RISK_GAIN in trading_core.metta from 6.0 to around 12.0 so the agent perceives the danger at the right scale. This is a single named constant and the comment above it in the code explains what it does.

Making a scenario like ours. The included realistic scenario was generated once with a fixed seed by the same script, so anyone can regenerate it and check it matches:

```
python3 make_scenario.py generate
```

Advantages

The first advantage is that the agent needs no trading rule at all. There is no line of code anywhere that says buy when the price does this or sell when it does that. The behavior comes out of the framework, which means the same agent design would work on any environment you can describe through the stimulus adapter, and the trading knowledge lives in a handful of named perception constants instead of a strategy.

The second advantage is robustness where fixed rules break. The momentum baseline loses 36 percent in the whipsaw market because its trigger fires on every wobble. The agent holds through the same noise and keeps its capital, because an action only executes when both subsystems agree it is worth it, and small moves never clear that bar. The same patience makes it lose less in the bleeding market. Over the four scenarios together the agent finishes ahead, 433.54 against 412.92.

The third advantage is interpretability. Every step of the run is one log line carrying the action, the portfolio value, the dominant emotion, all four feeling intensities, valence, securing and the two overgoals. When the agent sells, you can look at that exact step and see the securing rise and the valence fall that caused it. The charts make this visible to anyone: on the realistic series

you can watch fear climb while the agent sits out the crash and happiness return when it re-enters. A fixed rule gives you a trade list; this agent gives you a reason for every trade.

The fourth advantage is that the emotions are real readouts, not decoration. The decisions and the feelings are computed from the same motivational state in the same cycle, which the counterfactual experiments confirm. Change only the perceived market and the same agent picks different actions. Change only the personality and the same market produces different trades, down to an all-cautious agent that never trades at all.

The fifth advantage is that the whole thing is reproducible and testable. The price series are fixed, every reported number reproduces exactly on every run and was verified on two machines, 45 unit tests run under the repository's own test runner, and the evaluation pipeline computes nothing outside PeTTa.

Limitations

The first limitation is that a well tuned fixed rule beats the agent on the markets that rule was made for. The baseline wins both smooth trending series, and on the vshape the entire gap comes from one step, because its one percent hair trigger exits on the first down day while the agent waits for real evidence. The agent's value is robustness across regimes, not dominance on every series.

The second limitation is calibration. The perception gains map percent moves to perceived danger, and the defaults are tuned for panic sized moves such as daily cryptocurrency data. On gentler data like weekly stock closes the agent under-reacts until the risk gain is raised to match the asset, which the real Apple experiment measured directly, 82.5 with defaults against 95.71 with the matched gain. Related to this, the agent never sells in a slow bleed, because three percent legs never reach panic level turbulence, which is why it rides the downtrend scenario all the way.

The third limitation is the coarse action space. The agent is always all in or all out of a single asset. There are no partial positions, no position sizing, and no multiple assets, so it cannot express moderate confidence except by holding.

The fourth limitation is that the evaluation is on fixed series. That is the right choice for deterministic tests, but it means the results are single numbers rather than distributions over many random markets, so there is no statistical claim yet, only demonstrated behavior on chosen scenarios.

The fifth limitation is that the action goal correlations and the perception mapping are modeling choices. They are named constants with comments explaining the reasoning, and the papers leave them open, but different choices would give a different agent. The emotion intensities also live in a narrow band around the dominance threshold, so the dominant label mostly switches between happy and neutral, and sad and angry rarely dominate with the current appraisal dynamics.

None of these limitations is hidden by the results. The scenarios where the agent loses are reported with the same detail as the ones where it wins, and the calibration finding is documented as a finding rather than tuned away.

Conclusion

This use case set out to answer one question: if a trading agent has no trading rule and every decision comes from the MetaMo framework, does sensible behavior come out, and what does it add over a fixed rule?

The answer is yes, and the results show what it adds. The agent buys rallies, sells panic days, waits out crashes and re-enters recoveries, all from appraisal, motivation, consensus and stability dynamics working together on a simple market perception. Against a momentum baseline in the same world it wins where fixed rules break, holding through the whipsaw market that costs the baseline 36 percent, and it finishes ahead over the four scenarios combined, 433.54 against 412.92. It loses on clean trends, where a tuned threshold is close to ideal, and this is reported as openly as the wins.

The emotions turned out to be more than a nice extra. Because the feelings and the decisions are computed from the same state in the same cycle, the charts show fear rising exactly when the agent steps out of a crash and happiness returning exactly when it re-enters, on generated markets and on real Apple prices alike. The counterfactual experiments confirm there is no hidden rule: the same market with a different personality produces different trades, down to an all-cautious agent that never trades at all.

Everything is reproducible. The framework was consumed unchanged, the use case only adds files, 45 tests pass under the repository's own runner, every number reproduces exactly on every run, and anyone can repeat the whole pipeline or point the agent at their own price data with one converter script.

The main open directions are known and small: partial position sizes, evaluation over many random markets instead of fixed ones, and matching the perception gains to the asset automatically instead of by hand. What the use case already demonstrates is the main point: a motivational framework can drive a complete decision loop in a domain it was never written for, and explain itself while doing it.